

task2.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[4] ✓ 3s

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score
# Load California Housing Dataset
housing = fetch_california_housing(as_frame=True)

# Convert to DataFrame
df = housing.frame

# Display first 5 rows
df.head()
# Shape of dataset
print("Dataset Shape:", df.shape)

# Dataset info
df.info()

# Statistical summary
df.describe()
```

task2.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[4] ✓ 3s

```
# Create new intelligent features
# Correct feature engineering
df["Rooms_per_Household"] = df["AveRooms"] / df["AveOccup"]
df["Population_per_Household"] = df["Population"] / df["AveOccup"]
df["Bedrooms_per_Room"] = df["AveBedrms"] / df["AveRooms"]

df.head()

# Check updated dataset

# Input features
X = df.drop("MedHouseVal", axis=1)

# Target variable
y = df["MedHouseVal"]

# Train-Test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training size:", X_train.shape)
print("Testing size:", X_test.shape)
# Apply Standard Scaling
scaler = StandardScaler()
```

```
task2.ipynb
File Edit View Insert Runtime Tools Help

[4] 3s
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
# Linear Regression
lr_model = LinearRegression()
lr_model.fit(X_train_scaled, y_train)

# Predictions
lr_pred = lr_model.predict(X_test_scaled)

# Evaluation
lr_rmse = np.sqrt(mean_squared_error(y_test, lr_pred))
lr_r2 = r2_score(y_test, lr_pred)

print("Linear Regression RMSE:", lr_rmse)
print("Linear Regression R2 Score:", lr_r2)
# Ridge Regression
ridge_model = Ridge(alpha=1.0)
ridge_model.fit(X_train_scaled, y_train)

# Predictions
ridge_pred = ridge_model.predict(X_test_scaled)

# Evaluation
ridge_rmse = np.sqrt(mean_squared_error(y_test, ridge_pred))
ridge_r2 = r2_score(y_test, ridge_pred)

print("Ridge Regression RMSE:", ridge_rmse)
```

```
task2.ipynb
File Edit View Insert Runtime Tools Help

[4] 3s
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted House Prices (Decision Tree)")
plt.show()
best_model = results.loc[results["RMSE"].idxmin()]
print("Best Performing Model:")
print(best_model)
plt.figure()
plt.bar(results["Model"], results["RMSE"])
plt.xlabel("Model")
plt.ylabel("RMSE")
plt.title("RMSE Comparison of Models")
plt.show()
plt.figure()
plt.bar(results["Model"], results["R2 Score"])
plt.xlabel("Model")
plt.ylabel("R² Score")
plt.title("R² Score Comparison of Models")
plt.show()
errors = y_test - dt_pred

plt.figure()
plt.hist(errors)
plt.xlabel("Prediction Error")
plt.ylabel("Frequency")
plt.title("Error Distribution (Decision Tree)")
plt.show()
plt.figure()
```

task2.ipynb

File Edit View Insert Runtime Tools Help

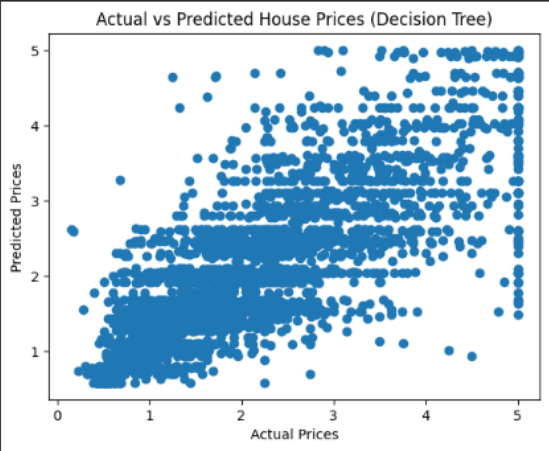
Commands + Code + Text Run all

pit.Show()

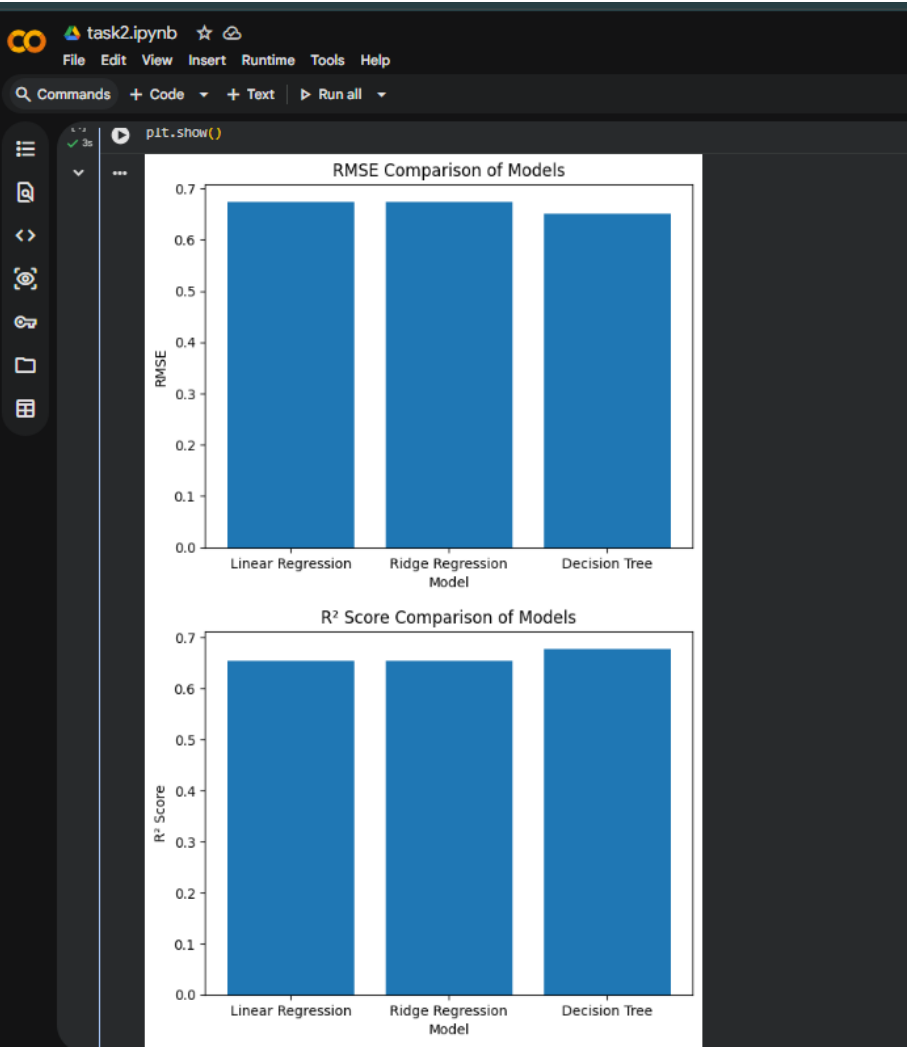
#	Column	Non-Null Count	Dtype
0	MedInc	20640 non-null	float64
1	HouseAge	20640 non-null	float64
2	AveRooms	20640 non-null	float64
3	AveBedrms	20640 non-null	float64
4	Population	20640 non-null	float64
5	AveOccup	20640 non-null	float64
6	Latitude	20640 non-null	float64
7	Longitude	20640 non-null	float64
8	MedHouseVal	20640 non-null	float64

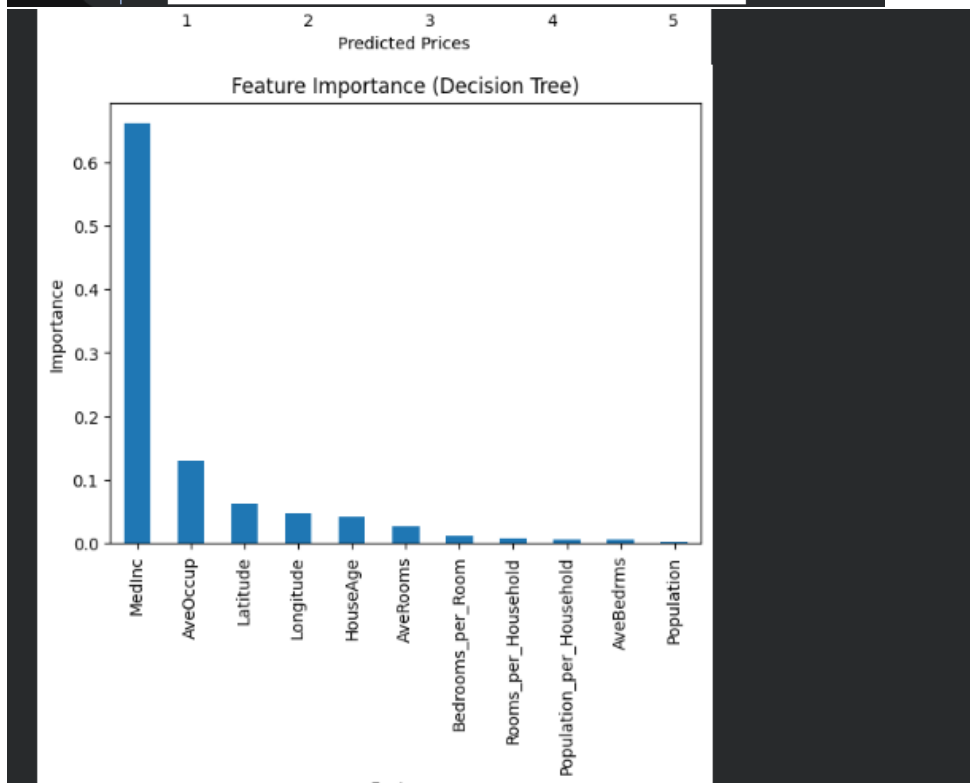
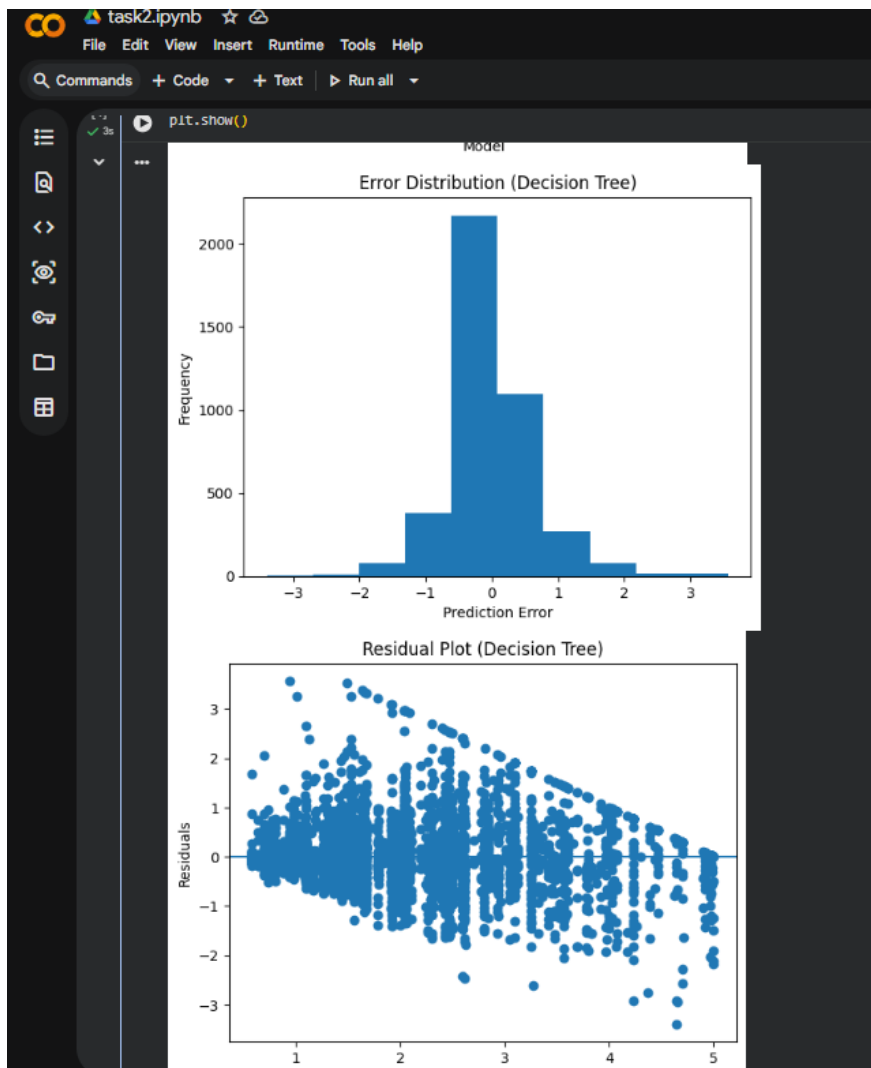
dtypes: float64(9)
memory usage: 1.4 MB
Training size: (16512, 11)
Testing size: (4128, 11)
Linear Regression RMSE: 0.6738172773395145
Linear Regression R2 Score: 0.6535205948827303
Ridge Regression RMSE: 0.6738201776284443
Ridge Regression R2 Score: 0.6535176121971285
Decision Tree RMSE: 0.649610532574982
Decision Tree R2 Score: 0.6779678321505103

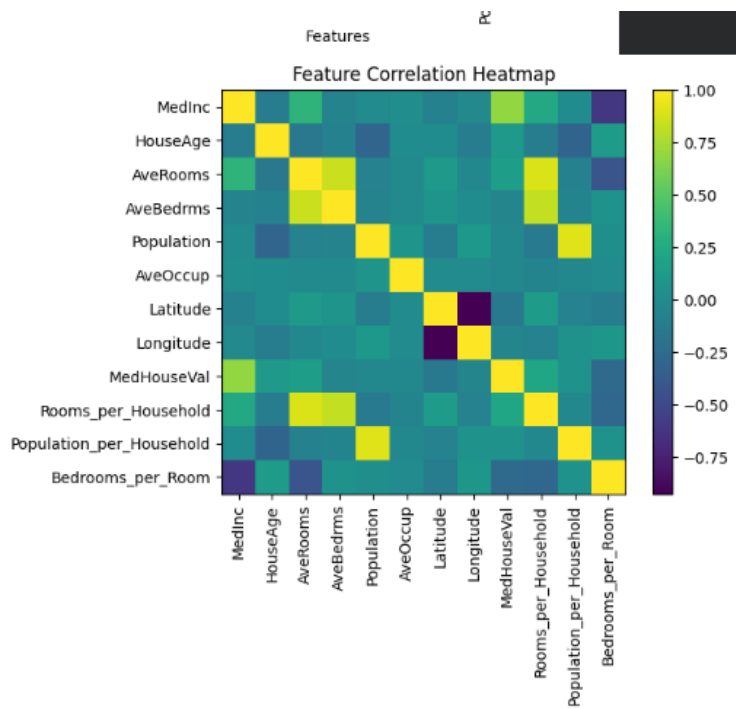
Actual vs Predicted House Prices (Decision Tree)



Best Performing Model:
Model Decision Tree
RMSE 0.649611
R2 Score 0.677968
Name: 2, dtype: object







Actual vs Predicted Prices (Sample Data)

